

PROYECTO DE SEGURIDAD EN APLICACIONES MÓVILES

— PIPELINE DevSecOps

Informe integrado: primer avance, segundo avance e informe final

Curso: CIB-204 · Seguridad del Software · Maestría en Ciberseguridad **Universidad:** CENFOTEC · Sede Central **Profesora:** MsC. Alejandra Corrales Vargas **Integrantes:** Eduardo J. Vega Arguedas y Keylor Elizondo Rodriguez (*confirmar*) **Repositorio:** <https://github.com/edovega/proyecto-devsecops-cib204> **Fecha:** (*completar*)

Mapa del documento

Parte	Secciones	Entrega
Parte I — Contexto y diseño	1–8	Primer Avance
Parte II — Implementación y entorno	9–12	Primer Avance
Parte III — Diagnóstico y remediación	13–18	Segundo Avance
Parte IV — Pruebas y riesgos	19–22	Segundo Avance
Parte V — Cierre académico	23–25	Informe Final

Índice de evidencias

ID	Evidencia	Archivo
EV-C204-001	Versiones del entorno (node, npm, docker)	docs/evidencias/capturas/EV-C204-001-v
EV-C204-002	Servicio /salud respondiendo	(pendiente)
EV-C204-003	Keycloak: realm appmovil	(pendiente)
EV-C204-004	Keycloak: cliente servicio-cifrado	(pendiente)
EV-C204-005	Keycloak: usuario demo	(pendiente)
EV-C204-006	App móvil cifrando/descifrando	(pendiente)
EV-C204-007	Pipeline Fase 1 (jobs en rojo)	(pendiente)
EV-C204-008	Reporte Semgrep (Fase 1)	(pendiente)
EV-C204-009	Reporte Gitleaks (Fase 1)	(pendiente)
EV-C204-010	SBOM (Fase 1)	(pendiente)
EV-C204-011	Informe ZAP (Fase 1)	(pendiente)
EV-C204-012	Alertas CodeQL (Code scanning)	(pendiente)
EV-C204-013	Pipeline Fase 2 (todo verde)	(pendiente)
EV-C204-014	Rama main protegida	(pendiente)
EV-C204-015	Figura 1. Contexto de la aplicación	docs/evidencias/figuras/figura-1-conte
EV-C204-016	Figura 2. Arquitectura y límites de confianza	docs/evidencias/figuras/figura-2-arqui
EV-C204-017	Figura 3. Flujo de cifrado con identidad	docs/evidencias/figuras/figura-3-flujo

PARTE I — CONTEXTO Y DISEÑO

1. Introducción

El presente proyecto desarrolla un **pipeline DevSecOps completo** para una aplicación móvil de cifrado de datos. La aplicación móvil actúa como cliente; el cifrado ocurre en un servicio backend desarrollado en **Node.js**; la identidad de los usuarios la gestiona **Keycloak** (IAM); y un pipeline de **GitHub Actions** ejecuta seis pruebas de seguridad automáticas en cada cambio del código.

La característica distintiva del laboratorio es que el proyecto **arranca inseguro a propósito**: contiene errores de seguridad reales (secretos quemados, dependencias vulnerables, validación deficiente de tokens, CORS abierto, entre otros), marcados en el código con comentarios `// [VULN-n]`. El pipeline los detecta automáticamente y el trabajo del estudiante consiste en corregirlos uno por uno hasta dejar todas las pruebas en verde.

La metodología sigue el ciclo **DevSecOps**: *detectar corregir volver a probar*. En la **Fase 1 (Diagnóstico)** se sube el proyecto tal cual y se documentan todos los hallazgos en rojo; en la **Fase 2 (Remediación)** se corrige cada error, se sube el cambio y se verifica la transición de rojo a verde. Este ciclo automatizado es el valor central del laboratorio: no basta con encontrar los errores, sino que el proceso garantiza que las correcciones se verifican de forma reproducible.

El marco de referencia del proyecto combina **NIST SSDF** (Secure Software Development Framework) para las prácticas del ciclo de vida seguro, **OWASP SAMM** para la madurez del proceso, **OWASP ASVS** y **OWASP MASVS** para los requisitos de seguridad verificables, y **STRIDE** para el modelado de amenazas.

2. Objetivos

2.1 Objetivo general

Implementar un pipeline DevSecOps completo para una aplicación móvil de cifrado —desde el análisis de requerimientos y el modelado de amenazas hasta el diagnóstico automatizado de vulnerabilidades y su remediación verificada—, dejando el pipeline en verde y documentando todo el proceso con evidencia.

2.2 Objetivos específicos

1. **Analizar los requerimientos de seguridad** de la aplicación móvil (confidencialidad, integridad, disponibilidad y autenticación), justificando cada uno y mapeándolos a estándares (OWASP SAMM, NIST SSDF, ASVS, MASVS).
2. **Modelar las amenazas** del sistema con STRIDE, identificando componente afectado, ejemplo de ataque, control y severidad por amenaza.
3. **Configurar el entorno de desarrollo en la nube** (GitHub Codespaces) con Node.js 20 y Docker, documentando la configuración paso a paso.
4. **Implementar el módulo de cifrado RSA** (padding OAEP) en el servicio backend, con funciones de utilidad de validación de entrada y manejo de errores.
5. **Configurar la identidad** con Keycloak (realm, cliente y usuario) y verificar el flujo completo de cifrado/descifrado con token.

6. **Ejecutar el pipeline de seis pruebas de seguridad** (SAST $\times 2$, secretos, SCA, imagen y DAST) y documentar los hallazgos de la Fase 1.
7. **Corregir las vulnerabilidades** en la Fase 2, verificando la transición de rojo a verde con trazabilidad por commit.
8. **Calcular los riesgos** de las vulnerabilidades con método probabilístico ($P \times I$) y análisis gerencial (ALE).
9. **Documentar todas las pruebas** realizadas (funcionales, del pipeline, pentest y automatizadas) con datos de entrada, esperado y real.
10. **Proteger la rama principal** y entregar el informe integrado en PDF con todas las tablas consolidadas.

3. Alcance, supuestos y reglas de compromiso

3.1 Alcance

- **App móvil (cliente):** aplicación Expo/React Native que cifra y descifra datos y maneja el token.
- **Servicio de cifrado (backend):** API Node.js con endpoints `/salud`, `/cifrar` y `/descifrar`, módulo RSA y validación de token.
- **IAM:** Keycloak que entrega y valida tokens JWT.
- **Pipeline DevSecOps:** seis pruebas automáticas en GitHub Actions.
- **Documentación:** informe integrado, tablas 1–6, evidencias y plantilla de diagnóstico.

3.2 Supuestos

- El entorno de ejecución es **GitHub Codespaces** (Node 20 y Docker preinstalados); no se instala nada en la computadora local.
- El repositorio es **privado durante el desarrollo** y se hará **público al finalizar** (requisito para que CodeQL sea gratuito y el docente pueda revisar).
- Los secretos del laboratorio son **ficticios, de práctica**; no se suben secretos reales.
- Mientras el repo esté privado, **Semgrep es la prueba SAST principal**; CodeQL no produce resultados hasta hacerlo público (según la guía).

3.3 Reglas de compromiso

- **Permitido:** ejecutar el laboratorio en el Codespace, generar y eliminar llaves de prueba, cifrar/descifrar datos de demostración, ejecutar el pipeline y descargar reportes.
- **Prohibido:** usar llaves o credenciales reales, subir secretos al repositorio, ejecutar el pentest contra sistemas de terceros, publicar el repositorio con datos sensibles.

4. Metodología

4.1 Ciclo DevSecOps

El proyecto sigue el ciclo **detectar corregir volver a probar**:

1. **Fase 1 — Diagnóstico:** se sube el proyecto inseguro, el pipeline se ejecuta solo y se documentan todos los hallazgos en rojo (Tabla 3).
2. **Fase 2 — Remediación:** se corrige cada hallazgo (un commit por corrección), se sube el cambio y se verifica la transición a verde (Tabla 4).

3. **Cierre:** pentest, cálculo de riesgos (Tabla 5), documentación de pruebas (Tabla 6), protección de la rama principal e informe final.

4.2 Marco de referencia

Marco	Aporte al proyecto
NIST SSDF (SP 800-218)	Prácticas del ciclo de vida seguro: definir requisitos (PW.4), implementar código seguro (PW.5–7), verificar (RV.1–3)
OWASP SAMM	Madurez del proceso DevSecOps: estrategia, diseño seguro, implementación, verificación, operaciones
OWASP ASVS	Requisitos de seguridad verificables (autenticación, validación, criptografía, errores, cabeceras)
OWASP MASVS	Requisitos específicos de aplicaciones móviles
STRIDE (Microsoft)	Modelado de amenazas por categoría
CWE	Identificación precisa de debilidades en los hallazgos

4.3 Reproducibilidad

Todo el entorno es reproducible: `.devcontainer/devcontainer.json` define el Codespace (Node 20 + Docker); `docker-compose.yml` define el stack (servicio + Keycloak); el pipeline se ejecuta en cada push. Los reportes se descargan como artefactos y se registran con hash SHA-256 en el manifiesto de evidencias.

5. Requerimientos de seguridad

Ver **Tabla 1** completa en <docs/tablas/tabla-1-requisitos.md>

Propiedad	Requisito	Control	Verificación
Confidencialidad	Los datos se cifran antes de viajar	RSA-2048 OAEP-SHA256 en <code>cifrado.js</code>	Roundtrip cifrar/descifrar; DAST
Integridad	Un dato alterado no se descifra	OAEP detecta manipulación; fallo seguro	Pentest: 1 byte alterado error controlado
Disponibilidad	El servicio no se cae con entradas raras	Validación de tipo y tamaño	Entrada vacía/100 MB/no texto rechazo
Autenticación	Solo usuarios válidos usan el servicio	JWT de Keycloak verificado en <code>auth.js</code>	<code>/cifrar</code> sin token 401; con token 200
No repudio (<i>extra</i>)	Toda operación queda registrada	Bitácora en <code>db.js</code>	Consulta de bitácora tras operar

Justificación: cada requisito se justifica en la Tabla 1 (por qué es necesario, qué ataque mitiga y qué estándar lo respalda). La justificación cubre: confidencialidad (red no confiable), integridad

(manipulación en tránsito), disponibilidad (DoS por entrada malformada), autenticación (abuso del servicio como oráculo de cifrado) y no repudio (auditoría).

6. Amenazas (STRIDE)

Ver **Tabla 2** completa en <docs/tablas/tabla-2-stride.md>

Amenaza	Componente	Ejemplo de ataque	Control	Severidad
Spoofing	API de cifrado	Usar el servicio sin identidad	JWT válido (Keycloak)	Alta
Tampering	Datos en tránsito	Alterar el texto cifrado	Descifrado que falla seguro	Alta
Repudiation	Servicio	Negar una operación	Bitácora y logs	Media
Information disclosure	Servicio / repo	Leer secretos o errores internos	Sin secretos; errores genéricos	Alta
Denial of service	API de cifrado	Enviar datos enormes	Límite de tamaño de entrada	Media
Elevation of privilege	Contenedor	Ejecutar como root	Contenedor no-root	Media

Cada amenaza incluye en la Tabla 2: severidad justificada, prueba concreta de verificación del control y CWE asociada (para cruzar con los hallazgos de la Tabla 3).

6.1 Casos de abuso (misuse cases)

Complementan el modelado STRIDE con escenarios concretos de uso malicioso:

ID	Caso de abuso	Actor	Resultado esperado del control
CA-01	Usar el servicio de cifrado sin autenticarse	Atacante externo	401 No autorizado
CA-02	Cifrar/descifrar con un token falso (firma inválida)	Atacante externo	401 No autorizado
CA-03	Alterar el texto cifrado en tránsito	Atacante MITM	Error controlado (OAEP detecta manipulación)
CA-04	Enviar entradas malformadas (vacías, enormes, no texto)	Atacante externo	Rechazo con 400; servicio sigue disponible
CA-05	Leer secretos del repositorio	Cualquiera (repo público)	Sin secretos en el repo (Gitleaks en verde)
CA-06	Explotar el servicio como oráculo de cifrado	Atacante autenticado	Rate limiting y bitácora (no repudio)

ID	Caso de abuso	Actor	Resultado esperado del control
CA-07	Escalar privilegios dentro del contenedor	Atacante con acceso	Contenedor no-root (Trivy en verde)
CA-08	Negar una operación realizada	Usuario	Bitácora consultable (no repudio)

7. Jerarquía de diseño

Ver [docs/jerarquia_diseno.md](#) — arquitectura, árbol de archivos, tabla de subsistemas y componentes críticos.

Figura 1. Contexto de la aplicación (EV-C204-015): la app móvil consume el servicio de cifrado (RSA), Keycloak valida la identidad y el pipeline DevSecOps revisa todo en cada cambio.

Figura 2. Arquitectura y límites de confianza (EV-C204-016): las tres capas (app, servicio, IAM) con sus límites de confianza.

Figura 3. Flujo de cifrado con identidad (EV-C204-017): la app pide un token a Keycloak, lo envía al servicio, y este valida el token antes de cifrar/descifrar con RSA.

7.1 Subsistemas

Subsistema	Función	Superficie de ataque	Propensión a fallar
App móvil	Cifrar/descifrar; maneja el token	Alta	Media
API de cifrado	Endpoints /salud, /cifrar, /descifrar	Alta	Alta
Módulo RSA	Cifrado/descifrado RSA-OAEP	Media	Media
Autenticación	Valida token JWT	Alta	Alta
Bitácora	Registra operaciones	Media	Media
IAM (Keycloak)	Entrega y valida tokens	Alta	Media
Pipeline	Pruebas de seguridad	Baja	Baja

7.2 Componentes críticos

Según los cuatro factores de la rúbrica (complejidad, frecuencia de uso, exposición a datos sensibles e interacción con componentes externos), los componentes críticos son: **API de cifrado**, **autenticación (JWT)**, **módulo RSA** y **app móvil**. Sus modos de fallo y mitigaciones están documentados en [docs/jerarquia_diseno.md](#).

7.3 Inventario de activos

ID	Activo	Tipo	Clasificación	Dueño	Almacenamiento
A-01	Texto plano del usuario	Datos	Confidencial	Usuario	En memoria, solo durante la operación
A-02	Texto cifrado (base64)	Datos	Confidencial	Usuario	En memoria / respuesta HTTP
A-03	Llaves RSA (pública/privada)	Criptográfico	Crítico	Servicio	Fuera del repo (env / secretos)
A-04	Tokens JWT	Credencial	Crítico	Usuario	En memoria de la app
A-05	Credenciales de Keycloak (admin/demo)	Credencial	Crítico	Administrador	Consola Keycloak / env
A-06	Bitácora de operaciones	Datos	Confidencial	Servicio	db.js (SQLite)
A-07	Código fuente (servidor + app)	Código	Público (al finalizar)	Equipo	Repositorio GitHub
A-08	Reportes de seguridad (Sengrep, Gitleaks, SBOM, ZAP)	Evidencia	Confidencial	Equipo	docs/evidencias/ + artifacts

El inventario alimenta el análisis ALE de la Tabla 5 (valor de activos y factor de exposición).

8. Configuración del entorno (paso a paso)

8.1 Entorno en la nube (Codespace)

1. Crear el repositorio público `proyecto-devsecops-cib204` (privado durante el desarrollo).
2. Abrir un Codespace en el repositorio (máquina 2-core). La configuración vive en `.devcontainer/devcontainer.json` (Node 20 + Docker).
3. Verificar las versiones:

```
node --version      # v20.x
npm --version       # 10.x
docker --version
docker compose version
```

Evidencia: EV-C204-001 (captura de versiones).

8.2 Levantar el servicio y Keycloak

```
docker compose up -d --build
docker compose ps      # ambos "running"
```

El servicio responde en el puerto 3000; Keycloak en el 8080. Verificación: abrir la URL del puerto 3000 con `/salud {"estado":"ok","version":"inseguro-1.0"}`.

Evidencia: EV-C204-002.

8.3 Configurar Keycloak (IAM)

1. Abrir la consola de administración (puerto 8080) con `admin/admin`.
2. Crear el realm `apppmovil`.
3. Crear el cliente `servicio-cifrado` (OpenID Connect, *Direct access grants*).
4. Crear el usuario `demo` (email, nombre, apellido, email verificado, contraseña no temporal).

Evidencia: EV-C204-003, EV-C204-004, EV-C204-005.

8.4 Probar la app móvil

```
bash iniciar-app.sh      # entra a app-movil/, instala y arranca Expo (puerto 8081)
```

El puerto 3000 debe estar en **Público** (Port Visibility Public) para que la app pueda llamar al servicio. En la app: pegar la URL del puerto 3000, escribir un texto, pulsar “Cifrar” y luego “Descifrar”.

Evidencia: EV-C204-006 (captura con “Cifrado OK” y “Descifrado OK”).

PARTE II — IMPLEMENTACIÓN Y ENTORNO

Nota de trazabilidad: esta parte documenta las **decisiones de diseño** del módulo de cifrado, las utilidades, la autenticación y el pipeline. El código fuente oficial (con sus vulnerabilidades // [VULN-*n*]) se integra desde el material del laboratorio; aquí se documenta el diseño seguro de referencia que guía la remediación de la Fase 2.

9. Módulo de encriptación/desencriptación (RSA)

9.1 Selección del algoritmo criptográfico

Decisión	Valor	Justificación
Algoritmo	RSA	Cifrado asimétrico estándar; el laboratorio lo fija
Tamaño de llave	2048 bits	Mínimo recomendado por NIST (SP 800-57); ~112 bits de seguridad
Padding	OAEP con SHA-256	OAEP es IND-CCA2 seguro; PKCS#1 v1.5 es vulnerable al ataque de oráculo de padding (Bleichenbacher, CWE-780)

Decisión	Valor	Justificación
Codificación de salida	Base64	Transporte seguro del texto cifrado (binario texto)

9.2 Funciones del módulo (cifrado.js)

Función	Entrada	Salida	Comportamiento seguro
<code>generarLlaves()</code>	—	par (pública, privada)	Genera RSA-2048; las llaves se guardan fuera del repo (nunca en el código)
<code>cifrar(textoPlano)</code>	texto	texto cifrado (base64)	Valida entrada; usa OAEP-SHA256; falla seguro ante error
<code>descifrar(textoCifrado)</code>	base64	texto plano	Valida entrada; OAEP detecta manipulación (falla con error controlado)

9.3 Diseño seguro de referencia vs. versión vulnerable

Aspecto	Versión insegura (Fase 1, esperada)	Diseño seguro (Fase 2)
Padding	PKCS#1 v1.5 o sin padding (CWE-780)	OAEP-SHA256
Llaves	Quemadas en el repo (CWE-798)	Variables de entorno / secretos de GitHub
Validación de entrada	Ausente o mínima	Tipo, tamaño y formato validados
Errores	Detalle interno en la respuesta (CWE-209)	Respuesta genérica; detalle en bitácora

La columna “versión insegura” se confirma con el código oficial del zip y los hallazgos reales de la Tabla 3.

10. Funciones de utilidad

10.1 Validación de entrada

- **Tipo:** solo cadenas de texto (rechaza números, objetos, `null`).
- **Tamaño:** límite superior (rechaza entradas enormes mitiga DoS, CWE-400).
- **Contenido:** sin caracteres de control problemáticos; codificación UTF-8.
- **Comportamiento:** rechazo con error genérico y código HTTP adecuado (400).

10.2 Manejo de errores

- Respuestas HTTP con mensajes **genéricos** (`{"error": "Error interno"}`) — nunca stack traces ni detalles de implementación (CWE-209).
- El detalle técnico se registra en la bitácora del servidor.
- **Fallo seguro:** ante cualquier excepción, la operación falla cerrada (no devuelve datos parciales).

10.3 Codificación y bitácora

- Base64 estándar para el texto cifrado.
- Bitácora de operaciones (quién, qué, cuándo) para el requisito de **no repudio** (Tabla 1).

11. Autenticación con Keycloak (JWT)

11.1 Flujo de identidad

1. La app móvil solicita un token a Keycloak (realm `appmovil`, cliente `servicio-cifrado`, usuario `demo`).
2. La app envía el token en el encabezado `Authorization: Bearer <token>`.
3. El servicio valida el token en `auth.js` **antes** de cifrar/descifrar.
4. Token válido 200 OK; token inválido/ausente 401.

11.2 Validación del token (diseño seguro)

Verificación	Qué valida	Cómo
Firma	El token fue emitido por Keycloak	<code>jwt.verify()</code> con la clave pública (JWKS)
Algoritmo	Solo RS256 (evita confusión de algoritmo)	<code>algorithms: ['RS256']</code> explícito
Expiración	El token no está vencido	Claim <code>exp</code>
Emisor	El token viene del realm correcto	Claim <code>iss</code> = realm <code>appmovil</code>
Audiencia	El token es para este servicio	Claim <code>aud</code> = cliente <code>servicio-cifrado</code>

La versión vulnerable esperada valida mal el token (por ejemplo, solo `jwt.decode()` sin verificar firma, o sin revisar `exp/iss`). La corrección sigue el **Patrón 4** del playbook de remediación.

12. Pipeline DevSecOps (las 6 pruebas)

12.1 Diseño del pipeline

Job	Tipo	Herramienta	Qué revisa	Artefacto
<code>sast_semgrep</code>	SAST	Semgrep	eval, inyección, JWT, CORS, errores	reporte Semgrep

Job	Tipo	Herramienta	Qué revisa	Artefacto
sast_codeql	SAST	CodeQL	Análisis con motor de GitHub (requiere repo público)	alertas en Code scanning
secretos	Secretos	Gitleaks	Claves y contraseñas quemadas	reporte Gitleaks
sca	SCA	npm audit + Trivy	Dependencias vulnerables	SBOM
imagen	Contenedor	Trivy + Syft	Fallas de imagen; SBOM	SBOM
dast	DAST	OWASP ZAP	Cabeceras, CORS	reporte-zap

12.2 Comportamiento por fase

- **Fase 1:** los jobs sast_semgrep, secretos, sca e imagen **fallan** (rojo) por las vulnerabilidades del código; CodeQL genera alertas (no falla el job); ZAP produce el informe de cabeceras/CORS.
- **Fase 2:** tras cada corrección (un commit por hallazgo), el job correspondiente pasa a **verde**; al final, 6/6 en verde y las alertas de CodeQL revisadas.

12.3 Configuración del workflow

- Se ejecuta en cada **push** a **main** (y en PRs).
- Los reportes se suben como **artifacts** (actions/upload-artifact) para descargarlos como evidencia.
- El workflow provisional está en `.github/workflows/devsecops.yml`; se reemplaza por la versión oficial del zip cuando esté disponible.

PARTE III — DIAGNÓSTICO Y REMEDIACIÓN

13. Fase 1 — Diagnóstico (pipeline en rojo)

(Pendiente de ejecución — se documenta cuando el zip esté integrado y el pipeline corra.)

Evidencia: EV-C204-007 (jobs en rojo), EV-C204-008 a EV-C204-012 (reportes).

Ver **Tabla 3** completa en [docs/tablas/tabla-3-diagnostico.md](#)

14. Fase 2 — Remediación (de rojo a verde)

(Pendiente de ejecución — el playbook de remediación está en [docs/remediacion-playbook.md](#).)

Ver **Tabla 4** completa en [docs/tablas/tabla-4-evidencias.md](#)

15. Protección de la rama principal

*(Pendiente — branch protection en **main**: requerir PR con revisión y status checks de los 6 jobs.)*

Evidencia: EV-C204-014.

PARTE IV — PRUEBAS Y RIESGOS

16. Cálculo de riesgo

Ver **Tabla 5** completa en [docs/tablas/tabla-5-riesgo.md](#)

Riesgo = Probabilidad × Impacto (escala 1–3), con análisis gerencial ALE.

17. Documentación de pruebas

Ver **Tabla 6** completa en [docs/tablas/tabla-6-pruebas.md](#)

18. Plantilla de diagnóstico del curso

Ver [docs/plantilla-diagnostico.md](#)

PARTE V — CIERRE ACADÉMICO

19. Resumen de resultados

(Pendiente — vulnerabilidades identificadas, riesgos asociados y efectividad de las medidas.)

20. Conclusiones

(Pendiente — evaluación de la seguridad y lecciones aprendidas del ciclo DevSecOps.)

21. Recomendaciones

(Pendiente — medidas adicionales: rotación de llaves, TLS, rate limiting, monitoreo, Dependabot, política SCA.)

22. Referencias

- Helfrich, J. (2019). *Security for Software Engineers*. Taylor & Francis Group.
 - Johnsson, D., Deogun, D. y Sawano, D. (2019). *Secure by Design*. Manning Publications.
 - Hoffman, A. (2020). *Web Application Security: Exploitation and countermeasures for modern web applications*. O'Reilly Media.
 - NIST. (2021). *Secure Software Development Framework (SSDF), SP 800-218*.
 - OWASP. *Software Assurance Maturity Model (SAMM)*.
 - OWASP. *Application Security Verification Standard (ASVS)*.
 - OWASP. *Mobile Application Security Verification Standard (MASVS)*.
 - Microsoft. *STRIDE threat model*.
 - MITRE. *Common Weakness Enumeration (CWE)*.
-

Anexos

- **Anexo A:** Tablas 1–6 completas
- **Anexo B:** Reportes del pipeline (Semgrep, Gitleaks, SBOM, ZAP, CodeQL)
- **Anexo C:** Manifiesto SHA-256 de evidencias
- **Anexo D:** Historial de versiones del repositorio
- **Anexo E:** Cronograma del proyecto
- **Anexo F:** Guía de lectura (evidencias, siglas y archivos)
- **Anexo G:** Glosario y convenciones
- **Anexo H:** Nota de transparencia IA

Anexo E — Cronograma del proyecto

Semana	Actividad	Entregable	Estado
1	Lectura de la guía y la rúbrica; creación del repositorio	Repo + estructura	
2	Datos del curso, plan de acción, validación de objetivos	docs/	
3	Primer Avance (10%): contexto, diseño, Tablas 1–2, entorno	Informe Parte I–II	(sin zip)
4–5	Integración del material oficial; Fase 1 (pipeline en rojo)	Tabla 3 + reportes	espera zip
6–7	Fase 2: remediación (un commit por hallazgo)	Tabla 4 + pipeline verde	
8	Pentest, riesgos (Tabla 5), pruebas (Tabla 6)	Tablas 5–6	
9	Segundo Avance (15%): diagnóstico, remediación, pruebas	Informe Parte III–IV	
10–13	Protección de rama, revisión de CodeQL, informe final	Informe Parte V	
14	Informe Final (15%): PDF integrado + repo público	Entrega	

Anexo F — Guía de lectura (evidencias, siglas y archivos)

Sistema de evidencias

Cada evidencia tiene un ID único **EV-C204-XXX** (ver índice de evidencias, sección inicial). Las evidencias de captura viven en `docs/evidencias/capturas/` y las figuras en `docs/evidencias/figuras/`. El manifiesto `docs/evidencias/manifiesto-sha256.txt` registra el hash SHA-256 de cada archivo de evidencia para garantizar integridad.

Siglas usadas

Sigla	Significado
SAST	Static Application Security Testing
DAST	Dynamic Application Security Testing
SCA	Software Composition Analysis
SBOM	Software Bill of Materials
IAM	Identity and Access Management
JWT	JSON Web Token
OAEP	Optimal Asymmetric Encryption Padding
CWE	Common Weakness Enumeration
STRIDE	Spoofing, Tampering, Repudiation, Information disclosure, DoS, Elevation
ALE	Annualized Loss Expectancy
SLE / ARO	Single Loss Expectancy / Annualized Rate of Occurrence

Archivos clave

Archivo	Qué es
<code>docs/informe-maestro.md</code>	Informe integrado (este documento)
<code>docs/tablas/tabla-1..6</code>	Tablas de entrega del laboratorio
<code>docs/validacion-objetivos.md</code>	Trazabilidad objetivos dónde cómo excede
<code>docs/remediacion-playbook.md</code>	Correcciones de la Fase 2
<code>.github/workflows/devsecops.yml</code>	Pipeline de 6 pruebas
<code>.devcontainer/devcontainer.json</code>	Configuración del Codespace

Anexo G — Glosario y convenciones

Término	Definición
Fase 1 (Diagnóstico)	Subir el proyecto inseguro; documentar los hallazgos en rojo del pipeline
Fase 2 (Remediación)	Corregir cada hallazgo con un commit; verificar la transición a verde

Término	Definición
Hallazgo	Vulnerabilidad detectada por una prueba del pipeline (con CWE)
Riesgo	Probabilidad × Impacto (escala 1–3)
VULN-n	Marcador de vulnerabilidad en el código oficial (// [VULN-n])
Rojo / Verde	Estado de un job del pipeline: falla / pasa

Convenciones: - Un commit por corrección, con mensaje descriptivo en español. - Los reportes se descargan como artefactos y se registran en el manifiesto SHA-256. - El repositorio es privado durante el desarrollo y público al finalizar. - Los secretos del laboratorio son ficticios; nunca se suben secretos reales.

Anexo H — Nota de transparencia IA

Este proyecto se desarrolló con asistencia de herramientas de IA (asistente de codificación en el entorno OpenWork) para: estructurar la documentación, redactar el informe y las tablas, generar las figuras, y agilizar tareas repetitivas del pipeline. **Todas las decisiones de seguridad, la ejecución del laboratorio, la verificación de resultados y el análisis crítico fueron realizados y validados por los integrantes del equipo.** El código fuente del laboratorio es el material oficial del curso; las correcciones de la Fase 2 fueron revisadas y verificadas por el equipo antes de cada commit.